

Analysis of Evolutionary Algorithms for Optimisation

Shvet Maharaj

27 August 2026

Table of contents

Introduction	2
Literature Review	2
Methodology	3
Design	3
Update procedures	3
Updating equations	3
Hessian methods	3
Gradient methods	3
Function value methods	4
Analysis	7
Measuring Performance	9
Results	11
Conclusion	11
Bibliography	12

Introduction

Literature Review

Optimisation can be thought of as a tool-selection problem. With different optimisation paradigms being the tools in your toolbox. A certain optimisation problem may be placed in front of you, the properties of which will inform which tools would suit the application. Each tool would have their unique set of tradeoffs — you *could* use a sledgehammer to nail together a wooden frame, and using a ball-peen hammer to take down a wall is certainly... possible, though it clearly would be more exhausting to tackle each of those problems in such a way.

With optimisation, ‘No Free Lunch’ theorems inform us that finding an optimisation strategy that is suited to a certain problem is how we gain benefits between different optimisation paradigms — “any elevated performance over one class of problems is offset by performance over another class” (Wolpert and Macready 1997).

It is typical that gradient methods perform better than function-value methods; with hessian methods performing even better. Although it is important to note that those methods do depend on being able to calculate or approximate gradients or Hessians for the optimisation problem at hand.

For optimisation problems where Hessians and gradients can be found, and where they are *useful* to optimisation algorithms (as opposed to being unstable to calculate, deceptive/misleading, or otherwise non-informative), the use of algorithms which can take advantage of them would be *expected* to be superior.

For those cases where usage of either gradients or Hessians are impossible or otherwise unfeasible, function-value (or direct) methods are available. While this flexibility in which optimisation problems they can be applied to is beneficial, function-value methods losing the information available from gradients or Hessians would either have no effect or a detrimental effect on an algorithm’s performance in optimisation. I.e., it would not be useful to use such a method when ones better suited to a problem exist. For this reason function-value methods like genetic algorithms are most informatively compared with fellow function-value methods.

This paper will primarily discuss the difference between genetic algorithms and their closest competitors: derivative-free optimisation algorithms, or black-box methods.

Methodology

Design

Update procedures

Where noted, a *strictly improving update procedure* will be used alongside the updating equations, defined as:

$$\mathbf{x}_{k+1} = \begin{cases} \mathbf{v}_{k+1} & \text{if } f(\mathbf{v}_{k+1}) < f(\mathbf{x}_k), \\ \mathbf{x}_k & \text{otherwise.} \end{cases}$$

Where $\mathbf{x}_k$ is the best performing point at iteration k , and $\mathbf{v}_{k+1}$ is the candidate point for the next iteration, $k + 1$. This candidate point is generated by the **updating equation**.

Alternatively, the update procedure could be a simpler, *direct updating procedure*, where every updated point, $\mathbf{v}_{k+1}$, is taken as the next iteration (thus the algorithm remains the same, but every $\mathbf{x}_k$ in the above formula is replaced with $\mathbf{v}_k$). No comparison with previous points is done. This would likely fit the deterministic algorithms better. The best solution at point k , $\mathbf{x}_k$, is stored separately as a vector – never accessed by the algorithm.

Updating equations

At the core of this exploration of genetic algorithms and other methods for optimisation, are the *updating equations*. These define how each optimisation paradigm attempts to iteratively improve a candidate solution, $\mathbf{v}$.

Hessian methods

Methods which utilise exact or approximate Hessians.

Newton-Raphson

Calculates the Hessian.

$$\mathbf{x}_{k+1} = \mathbf{v}_k - \frac{f'(\mathbf{v}_k)}{f''(\mathbf{v}_k)}$$

Gradient methods

Methods which utilise exact or approximate gradients.

Gradient descent

$$\mathbf{v}_{k+1} = \mathbf{v}_k - \nu \nabla f(\mathbf{v}_k),$$

where ν is the learning rate.

Function value methods

Also known as Direct Search, Pattern Search, or Black-Box Methods. These algorithms do not make use of gradient nor of hessian information. This makes them far easier to apply to any context.

Random search (RS)

The specific version of the random search algorithm is informed by the descriptions given in Zabinsky (2011). Algorithms are described by being ‘global’ or ‘local’.

Pure Random Search (PRS) repeatedly performs a global search with a simply improving update procedure.

The updating equation is defined as,

$$\mathbf{v}_{k+1} = \mathbf{u}.$$

It is notable that neither of the terms $\mathbf{x}_k$ nor $\mathbf{v}_k$ – the best or candidate points found by or for iteration k – are present in this formula.

Each generated point is *independent* of any previous point. The major benefit of this is PRS is ‘embarrassingly parallelisable’. This is the only optimisation algorithm that will have this property in this paper.

We test also an additional variant of RS, where the global search is either performed solely for the starting point), $\mathbf{x}_0$, or skipped entirely in favour of a user-defined starting point. Subsequent points are generated locally from the hypersphere (referred to as the ‘ $(N - 1)$ -sphere’, or ‘surface of the N -ball’) around $\mathbf{x}_k$.

This is Rastrigin’s **Fixed Step Size Random Search (FSSRS)**(Rastrigin 1963).

An efficient way to sample uniformly from the surface of an N -ball is given by Marsaglia (1972).

$$\mathbf{v}_{k+1} = \mathbf{x}_k + h\mathbf{u}.$$

Here, $\mathbf{u}$ is a normalised N -dimensional vector sample from the standard normal distribution such that,

$\mathbf{u} = \frac{1}{\|\mathbf{z}\|}\mathbf{z}$ and $\mathbf{z} \sim \mathcal{N}_N(0, 1)$, and where h is a step size hyperparameter.

Genetic algorithms

Genetic algorithms are a subset of evolutionary algorithms, typified by the use of three genetic operators: two-parent **crossover**, fitness-based **selection**, and random **mutation**.

Real-coded (in reference to $\mathbb{R}$, the real number set) genetic algorithms are used in this paper for greater applicability to arbitrary problems and models – despite genetic processes occurring with a discrete set of genes (ACGT) in nature.

Notation does shift here, due to the genetic algorithm necessarily requiring multiple candidates for each iteration. Our previous notation referencing a single candidate per iteration is thus no longer feasible.

We are implementing genetic algorithms using the **GA** package, as described in Scrucca (2013).

A brief explanation of the updating procedure follows.

Fitness evaluation

Fitness for each of the candidate solutions (‘genotypes’), $\theta_i^{(k)}$ is calculated, where i is an arbitrary index of the candidate solution in the generation k . This is typically the objective function of interest, $f(\cdot)$.

Selection

A subset of candidate solutions from the current generation’s population becomes selected as a separate population known as the ‘reproducing population’. Solutions are selected (with replacement) through a weighted selection process, based typically proportionally on the fitness of each genotype, $f(\theta_i^{(k)})$, with probability $p_i^{(k)}$.

An elitist strategy can be used, which means that even if the best performers are unluckily not selected, an amount of them are still are pushed through to the next population. The default method for real-valued parameters is **gareal_lsSelection**: proportional selection with fitness linear scaling.

Selection with linear scaling: `gareal_lsSelection()`

Let $\mathbf{f}$ contain the fitness values of each member in the current population.

To ensure all fitness values are positive:

$$\mathbf{f} \leftarrow \begin{cases} \mathbf{f} - \min(\mathbf{f}) & \text{if } \min(\mathbf{f}) < 0, \\ \mathbf{f} & \text{otherwise.} \end{cases}$$

Then, $\mathbf{f}$ is scaled as follows.

Let the mid-range, $M = \frac{\min(\mathbf{f}) + \max(\mathbf{f})}{2}$.

$$\delta = \begin{cases} \max(\mathbf{f}) - \bar{f} & \text{if } \bar{f} > M, \\ \bar{f} - \min(\mathbf{f}) & \text{otherwise,} \end{cases}$$

$$a = \frac{\bar{f}}{\delta}$$

$$b = \frac{\bar{f}}{\delta} (\delta - \bar{f}) = \bar{f} \left(1 - \frac{\bar{f}}{\delta}\right)$$

$$\mathbf{f}' = a\mathbf{f} + b.$$

Members of the population are then repeatedly sampled from, with probabilities proportional to their scaled fitness values, $\mathbf{f}'$, where:

$$p_i = \frac{f'_i}{\sum_j^N f'_j}.$$

Note that this is not the exact algorithm as contained within Scrucra's GA package. I have taken the liberty to simplify and rearrange terms for clarity of analysis.

Crossover

The core of a genetic algorithm, the crossover operation, brings a means to generate additional 'child' (next generation) genotypes through parts of the 'genetic information' (parameter values) from either of the 'parent' (current generation) genotypes. Alongside mutation, crossover applied on the reproducing population creates the new, next generation population. By default, `gareal_laCrossover` is used, which is local arithmetic crossover.

Local arithmetic crossover: `gareal_laCrossover()`

Algorithm explanation:

Mutation

The randomly shifting of the values of 'genes' (parameters) within genotypes. The probability of mutation can be set by the user. Alongside crossover, mutation applied on

the reproducing population creates the new, next generation population. By default, `gareal_raMutation` is used, which is uniform random mutation.

Uniform random mutation: `gareal_raMutation()`

Algorithm explanation:

Crossover and mutation operations increase diversity in a population, allowing us to explore more of the sample space (exploration), while selection operations decrease diversity in favour of increased fitness (exploitation).

Analysis

Traces of each algorithm for 100 iterations on a 2-dimensional parabolic test surface.

Coloured dots show the progression of the best point found by each algorithm.

White crosses show each point checked by the algorithm.

Note that these traces are intended to be viewed as an animation. These are accessible in [the Github repository for this project](#).

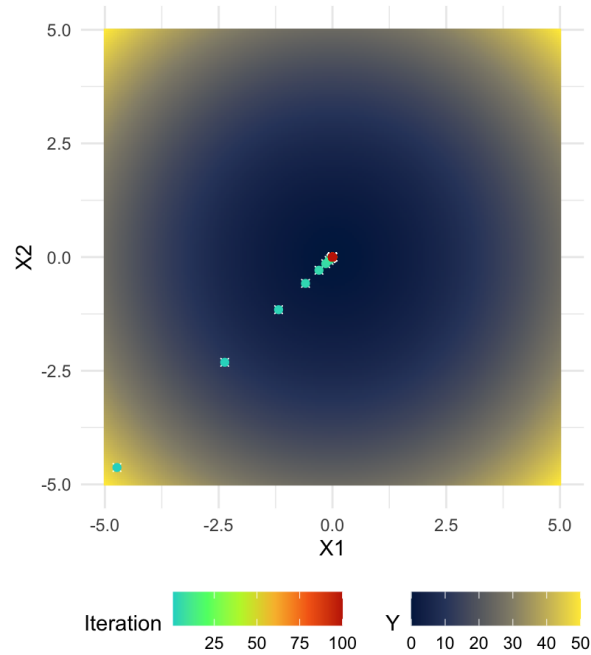


Figure 1: Newton-Raphson with $\nu=0.5$.

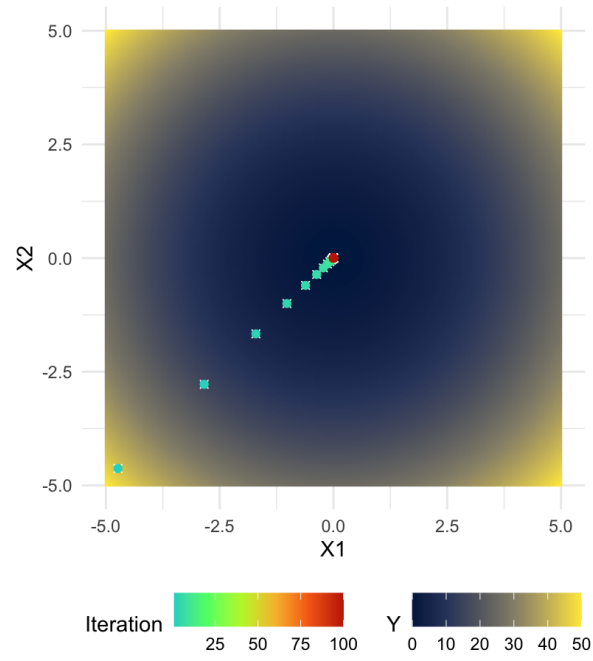


Figure 2: Gradient descent with $\nu=0.2$.

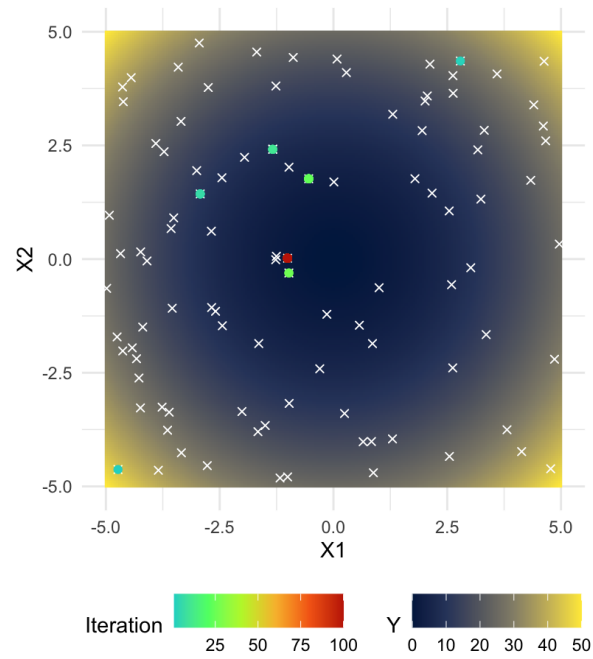


Figure 3: Pure random search.

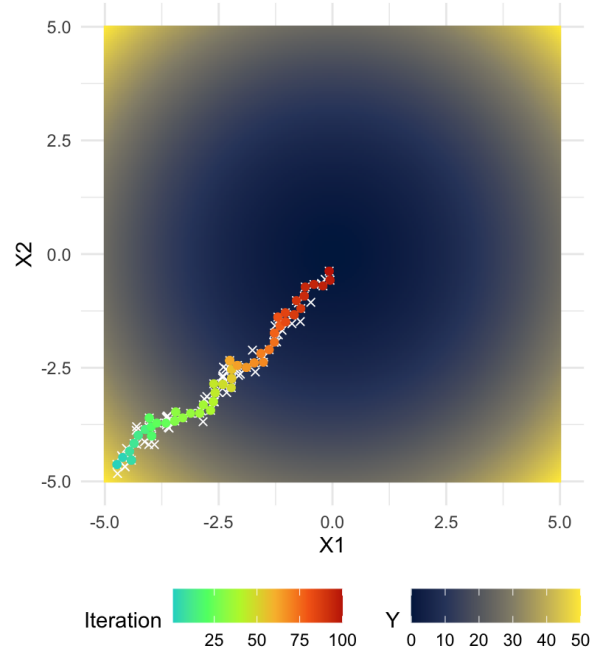


Figure 4: Fixed step size random search with $\nu=0.2$.

Measuring Performance

An interesting issue with drawing comparisons between GAs and optimisation procedures is the issue of fairly assessing the amount of function calls. We could simply compare the iteration number like we have with the other optimisation procedures. We would have to note how some procedures require more overhead for each function call, but until now, these overheads above the function call were not too excessive. Now with GAs, we have an entire population being assessed with each iteration. The GA package (Scrucca 2017) uses a default population size of 50, and throws a warning if the population size is less than 10.

A solution to this is taking the product of the iteration number and population size to get a value of function calls completed by the process. This does seem to be the fairest assessment method. When attempting this, we might use the iteration limit of 100 that we have currently been using, and divide that by the population size being used.

Whoops. Using the default value, we've reached our maximum amount of function calls in two population iterations.

Clearly, we have not given GAs a fair chance to flex their muscles. There has barely been enough time for any of the 'learning' that GAs are capable of to come through in a single step.

So perhaps we try a more balanced value: 10 iterations of population size 10.

Ah phooey. We now have the issue of rarer events like mutation ($p = 0.1$) only occurring once on average per population step. Additionally, 10 population steps is still squeezing the algorithm quite a bit. This seems hardly fair. Random search will have trouble making any meaningful progress towards a solution if it was given 10 iterations. Even gradient methods would have the issue of having suboptimal results with 10 iterations. Therefore, this is clearly not an issue that lands solely with direct search methods.

Perhaps we could consider *unique* function calls. This may be a better fit. GAs do have a tendency to repeatedly check the same values. Well-performing genes will survive to the next generation, to be tested again. Crossover does complicate this, but it should be clear that in principle, the probability of collisions is increased through this form of performance sampling. Though this does diminish quickly as the dimensions of the search increase.

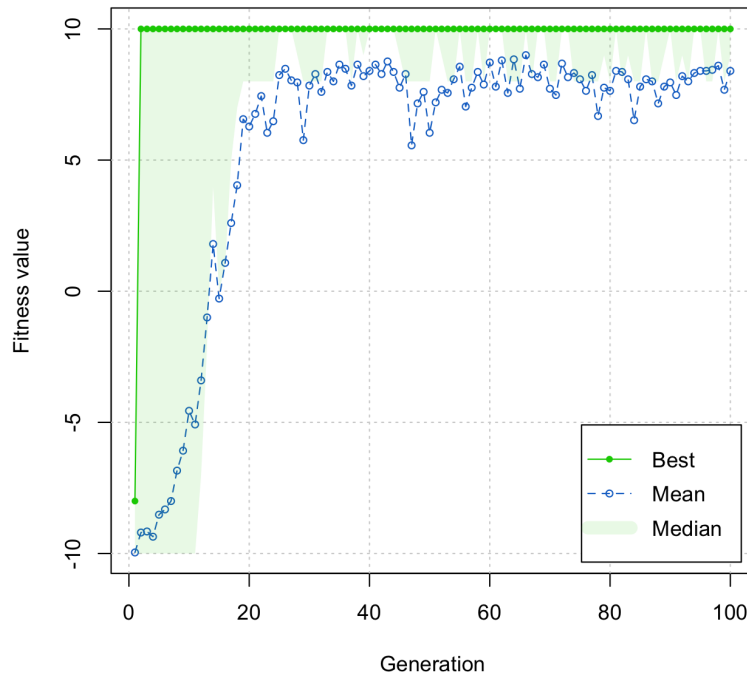


Figure 5: ‘Drone game’ performance of GA with population size 50 over 100 population iterations.

Wow! Clearly GAs do a pretty fine job. In just 2 population steps we had already had at least one full set of 10 drones successfully completing the course. Additionally by iteration 25, even the mean score of the drones is comfortably sitting at ~ 8 . Noting that minimum score possible is -10, and maximum is 10, that means we consistently have 9/10 drones completing the course.

So why have I been so negative toward GA, when its performance looks so promising?

Well, let's consider some things that these graphs are not showing. 2 steps means between 51-100 function calls, the GA was able to eventually gain its first completely successful run. This is not completely bogus, it's a very fair result from a direct search algorithm. This is a comfortable position to be in relation to RS.

This took approximately xyz seconds to run. In comparison, a local random search algorithm takes zyx seconds. This is a abc -fold increase over other methods. When we compare the performance of the RS training of the neural network, we find that:

Even if we stop the training after 25 population iterations, that is still 1250 function calls.

Results

Conclusion

Bibliography

- Marsaglia, George. 1972. “Choosing a Point from the Surface of a Sphere.” *The Annals of Mathematical Statistics* 43 (2): 645–46. <https://doi.org/10.1214/aoms/1177692644>.
- Rastrigin, L. A. 1963. “The Convergence of the Random Search Method in the Extremal Control of a Many Parameter System.” *Automaton & Remote Control* 24: 1337–42. <https://cir.nii.ac.jp/crid/1573950398967802624>.
- Scrucca, Luca. 2013. “**GA**: A Package for Genetic Algorithms in *R*.” *Journal of Statistical Software* 53 (4). <https://doi.org/10.18637/jss.v053.i04>.
- Wolpert, D. H., and W. G. Macready. 1997. “No Free Lunch Theorems for Optimization.” *IEEE Transactions on Evolutionary Computation* 1 (1): 67–82. <https://doi.org/10.1109/4235.585893>.
- Zabinsky, Zelda B. 2011. “Random Search Algorithms.” *Wiley Encyclopedia of Operations Research and Management Science*, ahead of print, January. <https://doi.org/10.1002/9780470400531.eorms0704>.